

软件设计模式 练习题集

(共 10 题: 5 道画图题 + 5 道填空题)

一、画图题 (UML 类图绘制)

题目 1 (工厂方法模式)

在线学习平台需要实现不同类型学习资源的创建与展示, 涉及课后作业、课堂笔记两种在线学习资源, 要求使用工厂方法模式设计该资源管理系统, 请绘制 UML 类图。

题目 2 (单例模式)

学校教务管理系统需要实现学生成绩单、补考通知单、学籍证明在线打印功能。系统如果创建多个打印服务对象, 会出现打印队列错乱、重复提交打印任务、打印机端口占用异常问题, 因此规定整个程序运行期间, 创建打印机服务对象。要求使用单例模式来设计这个打印服务类, 同时提供打印方法, 请绘制 UML 类图。

题目 3 (简单工厂模式)

某电商平台后台需要实现订单数据导出功能, 支持 Word 订单文档、CSV 表格文件两种格式导出。请采用简单工厂模式, 设计一个工厂类 (ExportFactory), 该工厂提供了 `createFileExport()` 方法, 该方法会根据不同参数创建对应文档导出类实例 (如 `WordExport`、`CsvExport`)。文档导出类都实现了文件导出接口 (`FileExport`)。请绘制该类图。

题目 4 (外观模式)

现开发一个企业智慧办公系统，需要通过统一门面类 OfficeFacade 提供两种场景模式：上班模式 workOnMode()、下班模式 workOffMode()。系统内部包含多个相互独立的子系统设备：照明灯(Light)、窗帘(Curtain)、门禁锁(DoorLock)。每一个设备子系统都各自拥有 turnOn() 开启、turnOff() 关闭方法，客户端不需要逐个操控单个设备，只调用门面类对应的模式方法即可一键批量联动所有设备，请使用外观模式完成整体设计。请绘制 UML 类图。

题目 5（对象适配器模式）

校园多媒体点播系统定义了统一视频播放器标准接口，规定系统内播放器必须遵循该接口规范完成播放操作。项目前期接入一款老旧第三方视频解码播放器，该第三方源码无法修改，其内部方法命名、调用方式和现有系统标准接口不一致。为在不修改原有第三方播放器代码、不改动现有系统业务代码的前提下，让老旧播放器适配接入点播系统，要求使用对象适配器模式完成接口兼容改造。请简述改造思路，并画出类图（可用 mermaid 代码表示）。

二、填空题（代码补充）

题目 1（工厂方法模式）

请补充代码：

```
// 抽象产品
public interface LearningResource {
    // 展示学习资源的方法
    void show();
}

// 具体产品
public class Homework implements ① _____ {
    @Override
    public void show() {
        System.out.println("工厂方法模式作业");
    }
}

// 具体产品
public class ClassNote implements LearningResource {
```

```

        @Override
        public void show() {
            System.out.println("工厂方法模式核心要点、类关系及代码实现");
        }
    }

// 抽象工厂
public interface ResourceFactory {
    // 创建学习资源对象
    LearningResource createResource();
}

// 课后作业工厂
public class HomeworkFactory implements ResourceFactory {
    @Override
    public LearningResource createResource() {
        // 创建并返回 Homework 对象
        return ② _____;
    }
}

// 课堂笔记工厂
public class ClassNoteFactory implements ResourceFactory {
    @Override
    public LearningResource createResource() {
        return new ClassNote();
    }
}

public class Client {
    public static void main(String[] args) {
        // 使用作业工厂创建课后作业
        ResourceFactory homeworkFactory = new HomeworkFactory();
        // 调用工厂对象 createResource 方法，创建课后作业资源对象
        LearningResource homework = ③ _____;
        homework.show();
    }
}

```

第 1 空:

第 2 空:

第 3 空:

题目 2（单例模式）

请补充代码：

```
class PrintService {
    //创建单例类的实例
    private static PrintService instance = ①_____；
    //构造函数私有化
    private ②_____() {}
    public static PrintService getInstance() {
        //返回单例类实例
        return ③_____；
    }
    public void print(String docName) {
        System.out.println("打印: " + docName);
    }
}

public class Test {
    public static void main(String[] args) {
        //获取单例类实例
        PrintService printService = ④_____；
        //调用 printService 对象的 print 方法执行打印任务，参数可以任意字符串
        ⑤_____；
    }
}
```

题目 3（简单工厂模式）

请补充代码：

```
// 抽象产品
public interface FileExport {
    void exportOrder(String orderNo);
}
```

```

// 具体产品
public class WordExport implements ① _____ {
    @Override
    public void exportOrder(String orderNo) {
        System.out.println("正在导出【Word 订单文档】");
        System.out.println("订单编号: " + orderNo);
        System.out.println("Word 文件生成完成\n");
    }
}

// 具体产品
public class CsvExport implements FileExport {
    @Override
    public void exportOrder(String orderNo) {
        System.out.println("正在导出【CSV 表格订单文件】");
        System.out.println("订单编号: " + orderNo);
        System.out.println("CSV 文件生成完成\n");
    }
}

// 工厂类
public class ExportFactory {
    public FileExport createFileExport(String type) {
        if ("word".equalsIgnoreCase(type)) {
            // 创建并返回 WordExport 实例
            return ② _____;
        } else if ("csv".equalsIgnoreCase(type)) {
            return new CsvExport();
        } else {
            throw new RuntimeException("不支持该导出格式: " + type);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        ExportFactory factory = new ExportFactory();
        FileExport wordExport = factory.createFileExport("word");
        // 导出 Word 订单, 订单号可以是任意字符串
        ③ _____;
    }
}

```

题目 4（外观模式）

请补充代码：

```
//照明灯
public class Light {
    public void turnOn() {
        System.out.println("客厅主灯已打开");
    }
    public void turnOff() {
        System.out.println("客厅主灯已关闭");
    }
}

//窗帘
public class Curtain{
    public void turnOn() {
        System.out.println("客厅窗帘已打开");
    }
    public void turnOff() {
        System.out.println("客厅窗帘已关闭");
    }
}

//门禁锁
public class DoorLock {
    public void turnOn() {
        System.out.println("门禁锁已开启");
    }
    public void turnOff() {
        System.out.println("门禁锁已关闭");
    }
}

//外观类
public class OfficeFacade {
    private Light light = ①_____；
    private Curtain curtain = ②_____；
    private DoorLock doorLock = ③_____；
    public void workOnMode() {
        System.out.println("====触发【上班模式】====");
    }
}
```

```

        light.turnOn();
        curtain.turnOn();
        doorLock.turnOff();
    }
    public void workOffMode() {
        System.out.println("====触发【下班模式】====");
        light.turnOff();
        curtain.turnOff();
        doorLock.turnOn();
    }
}

public class Client {
    public static void main(String[] args) {
        //创建外观对象
        OfficeFacade officeFacade = ④_____；
        //通过外观类对象，触发上班模式
        ⑤_____；
    }
}

```

题目 5（对象适配器模式）

请补充代码：

```

//目标接口
public interface MediaPlayer {
    void playVideo();
}
//被适配类
public class OldPlayer {
    public void play() {
        System.out.println("老旧播放器：正在解码视频并播放世界杯...");
    }
}
//适配器类，实现目标接口
public class VideoAdapter implements ①_____ {
    private OldPlayer oldPlayer;
    public VideoAdapter(OldPlayer oldPlayer) {

```

```
//初始化 oldPlayer
    ②_____;
```

```
}
```

```
@Override
public void playVideo() {
    System.out.println("适配器进行接口转换...");
    //调用 oldPlayer 对象的 play()方法播放视频
    ③_____;
```

```
}
```

```
}
```

```
public class Test {
    public static void main(String[] args) {
        OldPlayer oldPlayer = new OldPlayer();
        //创建适配器实例，该实例封装了 OldPlayer 实例
        MediaPlayer player = ④_____;
```

```
        //调用适配器对象的 playVideo()方法播放视频
        ⑤_____;
```

```
    }
}
```